Bezpieczeństwo supply-chain w chmurze: pipeline CI/CD z wersjonowanymi obrazami w ECR i rolling-deployment na AWS ECS z automatycznym rollbackiem

Bezpieczeństwo rozwiązań chmurowych

Gr. P1

Marcin Bator 173592
Adrian Gola 169542
Igor Litwa 173659

Spis treści

1. Koncepcja projektu 3
1.1 Problem i kontekst 3
1.2 Zakres 3
1.3 Punkt wyjścia 3
1.4 Cele projektowe 4
1.5 Założenia i wyzwania 4
1.6 Technologie i środowisko 5
1.7 Charakter pracy i planowane rezultaty 6

1. Koncepcja projektu

Rozdział opisuje problem, zakres i cele projektu oraz przyjęte założenia, technologie i planowane rezultaty. Kolejne rozdziały rozwijają architekturę, implementację i wyniki testów.

1.1 Problem i kontekst

W architekturach kontenerowych obraz Dockera jest artefaktem, któremu środowisko produkcyjne ufa w pełni. Jeśli atakujący podmieni obraz (nadpisze tag w rejestrze, użyje wykradzionych kluczy CI, wstrzyknie złośliwą zależność lub skompromitowaną akcję w pipeline), na produkcji uruchomi się obcy kod. Ataki typu SolarWinds, Codecov czy kompromitacje popularnych GitHub Actions pokazują, że to duże zagrożenie.

Drugim elementem łańcucha dostaw jest infrastruktura, na której działa obraz. Zmiana w Terraform może udostępnić zasób publicznie (bucket S3, otwarta grupa bezpieczeństwa, zbyt szeroka rola IAM) albo niezauważenie podnieść miesięczny koszt środowiska, np. przez dodanie NAT Gateway lub większej liczby zadań.

Trzeci problem to ciągłość działania. Wadliwa wersja aplikacji nie może wyłączyć usługi, także wtedy, gdy błąd ujawnia się dopiero kilka minut po wdrożeniu, pod rzeczywistym ruchem.

1.2 Zakres

Zakres projektu obejmuje:

- aplikację API w Pythonie (FastAPI) z endpointami /health i /version,
- Dockerfile (multi-stage, użytkownik non-root, przypięta wersja obrazu bazowego),
- infrastrukturę jako kod w Terraform,
- pipeline aplikacji w GitHub Actions: build, skan, SBOM, wersjonowanie, push do ECR, podpisanie, weryfikacja podpisu, deploy,
- pipeline infrastruktury w GitHub Actions: plan Terraform, skan konfiguracji, szacowanie kosztu zmian z progiem wymagającym ręcznej akceptacji, apply,
- wdrożenie blue/green z okresem obserwacji nowej wersji i automatycznym rollbackiem na podstawie alarmów CloudWatch.

1.3 Punkt wyjścia

Nie ma istniejących elementów, całość powstaje od zera w ramach projektu. Wykorzystane zostaną gotowe narzędzia open source (Terraform, Trivy, Infracost) i usługi AWS.

1.4 Cele projektowe

Postawiono trzy główne cele projektu. Ich priorytety i sposób sprawdzenia zestawiono w Tab. 1.1.

Priorytet	Cel	Sposób sprawdzenia
1	**Bezpieczeństwo obrazów** Do produkcji trafia tylko obraz zbudowany przez pipeline, przeskanowany, z SBOM i podpisany. Usługa wdrażana jest po digeście, a podpis jest weryfikowany przed jej aktualizacją.	Pipeline kończy się błędem na obrazie ze znaną podatnością CRITICAL. Próba wdrożenia niepodpisanego obrazu zostaje odrzucona. SBOM jest dołączony jako artefakt.
2	**Bezpieczeństwo infrastruktury** Każda zmiana w Terraform przechodzi przez PR ze skanem konfiguracji i oceną kosztu. Zmiana podnosząca koszt ponad próg wymaga ręcznej akceptacji. Pipeline łączy się z AWS przez OIDC, a akcje są przypięte do SHA.	PR z publicznie dostępnym zasobem zostaje zablokowany. PR podnoszący koszt ponad próg nie zostaje wdrożony bez zatwierdzenia.
3	**Ciągłość działania i rollback** Nowa wersja dostaje ruch dopiero po przejściu healthchecków. Przez 10 minut po przełączeniu ruchu poprzednia wersja nadal działa, a alarm w tym czasie przywraca do niej ruch.	Obraz nieprzechodzący /health nie dostaje ruchu. Obraz, który zaczyna zwracać błędy kilka minut po wdrożeniu, zostaje automatycznie wycofany przy ciągłym ruchu testowym.

Tab. 1.1. Cele projektowe

W celu 3 rozróżniono dwie sytuacje. Gdy nowa wersja nie przechodzi healthchecków, nie przejmuje ruchu i wdrożenie zostaje przerwane, a usługę nadal obsługuje wersja bieżąca. Nie jest to rollback, bo nowa wersja nigdy nie została wdrożona. Rollback dotyczy wersji, która przeszła healthchecki i przejęła ruch, ale w okresie obserwacji (bake time) przekroczyła progi alarmów, np. udziału odpowiedzi 5xx lub czasu odpowiedzi. Wtedy ECS przełącza ruch z wersji 1.1 z powrotem na 1.0, która przez ten czas nadal działa.

1.5 Założenia i wyzwania

Przedmiotem projektu jest proces dostarczania aplikacji i infrastruktury (supply chain). Zakłada się jedno środowisko AWS, tworzone i usuwane przez Terraform. Repozytorium na GitHubie jest jedynym zaufanym źródłem kodu aplikacji i infrastruktury. Główne wyzwania to:

- Brak natywnej weryfikacji podpisów w ECS. Weryfikacja podpisu zostanie więc wykonana w pipeline przed aktualizacją usługi.

- Dobór progów healthchecków i alarmów. Zbyt agresywne progi powodują fałszywe rollbacki, a zbyt łagodne wydłużają wykrycie awarii. Okres obserwacji musi być dłuższy niż czas potrzebny alarmowi na zmianę stanu.
- Szacowanie kosztów. Infracost wycenia zasoby z planu Terraform według cennika AWS, ale koszty zależne od użycia (transfer, liczba żądań) wymagają przyjęcia założeń. Jako próg przyjęto wzrost szacowanego kosztu miesięcznego o więcej niż 10 USD względem gałęzi głównej. Po przekroczeniu progu krok apply trafia do środowiska GitHub z wymaganym zatwierdzeniem.
- Tag a digest. Digest jest znany dopiero po wypchnięciu obrazu, więc pipeline musi go odczytać i przekazać do nowej rewizji task definition. Konieczne jest też rozdzielenie odpowiedzialności: Terraform zarządza infrastrukturą, a pipeline aplikacji wersją obrazu, tak aby oba mechanizmy nie kolidowały.
- Sekrety i dane poufne. Repozytorium nie będzie zawierać rzeczywistych sekretów, a stan Terraform będzie przechowywany w zdalnym backendzie (S3 z blokadą).

1.6 Technologie i środowisko

Dobór technologii wynika z wymagań, aby całość była powtarzalna (IaC) i zautomatyzowana (CI/CD). Zestawienie technologii przedstawia Tab. 1.2.

Obszar	Technologia	Zastosowanie
Aplikacja	Python, FastAPI, Docker	Endpointy /health i /version, obraz multi-stage z użytkownikiem non-root i przypiętym obrazem bazowym
Infrastruktura jako kod	Terraform, backend S3	Moduły sieci, ECR, ECS, ALB, IAM/OIDC i monitoringu, odtworzenie i usunięcie środowiska jednym poleceniem
CI/CD	GitHub Actions, OIDC	Osobne workflow dla aplikacji i infrastruktury, logowanie do AWS bez długoterminowych kluczy
Rejestr obrazów	Amazon ECR	Skanowanie przy push, niezmienne tagi, polityka czyszczenia starych obrazów
Podpisywanie obrazów	Cosign, AWS Signer	Podpis obrazu w pipeline i weryfikacja przed wdrożeniem
Bezpieczeństwo obrazów	Trivy	Skan podatności blokujący CRITICAL, generowanie SBOM
Bezpieczeństwo infrastruktury	Trivy, Infracost	Skan konfiguracji Terraform, szacowanie kosztu zmian w PR z progiem akceptacji
Wdrożenie	Amazon ECS, ALB, CloudWatch	Blue/green z okresem obserwacji, alarmy 5xx i czasu odpowiedzi jako warunek rollbacku
Testy	Generator ruchu, AWS CLI, skrypty	Ciągły ruch podczas wdrożenia, scenariusze ataku i awarii

Tab. 1.2. Technologie i ich zastosowanie

1.7 Charakter pracy i planowane rezultaty

Projekt ma charakter wdrożeniowy, tzn. powstaje działające środowisko na AWS wraz z pipeline'ami dla aplikacji i infrastruktury. Zawiera również element porównawczy (Cosign i AWS Signer) oraz krótką analizę alternatywnych strategii wdrożenia (rolling deployment z circuit breakerem, natywny blue/green w ECS, blue/green przez CodeDeploy), która uzasadnia przyjęte decyzje.

Planowane rezultaty to:

- repozytorium z kodem aplikacji, Dockerfile, definicjami Terraform i workflow GitHub Actions,
- działające środowisko AWS prezentowane na żywo,
- diagram architektury oraz diagramy przepływu pipeline'u aplikacji i pipeline'u infrastruktury,
- wyniki testów: dostępność usługi podczas wdrożenia, automatyczny rollback, zablokowany obraz z podatnością CRITICAL, odrzucony niepodpisany obraz, zablokowany PR z błędną konfiguracją, PR oczekujący na akceptację kosztów,
- dokumentacja zgodna z wymaganiami.

Prezentacja zostanie oparta na demonstracji czterech scenariuszy:

1. Poprawny deployment. Nowa wersja przejmuje ruch bez przerwy w działaniu, co widać w ciągłym ruchu testowym i w endpointcie /version.
2. Wersja psująca się po wdrożeniu. Obraz przechodzi healthchecki i przejmuje ruch, ale po kilku minutach zaczyna zwracać błędy 5xx. Alarm w okresie obserwacji uruchamia rollback i ruch wraca do poprzedniej wersji.
3. Atak na supply chain. Próba wdrożenia niepodpisanego obrazu lub obrazu z podatnością CRITICAL zostaje zablokowana.
4. Kosztowna zmiana infrastruktury. PR dodający drogi zasób dostaje komentarz z szacunkiem kosztu, a apply czeka na ręczne zatwierdzenie.